

Rapport technique - Satisfaction Client

1. Objectif du projet

Le projet Satisfaction Client transforme des avis clients non structurés en indicateurs exploitables pour une entreprise. Il permet de collecter ou importer des avis, de les analyser avec un modèle de sentiment propriétaire, de restituer les résultats dans une application web et d'alimenter une boucle d'amélioration continue via des corrections humaines.

La solution répond à une problématique de satisfaction client en aval de la supply chain : comprendre rapidement les irritants liés à la livraison, au service client, au prix, au remboursement, à la qualité produit ou à la conformité des commandes.

2. Architecture générale

Le projet est orchestré avec Docker Compose et se compose de plusieurs services complémentaires.

frontend

Technologie. React, Vite, TypeScript

Role. Interface produit pour lancer, comparer et corriger les analyses.

api

Technologie. FastAPI

Role. API produit, endpoints métier protégés par JWT et rôles.

celery_worker

Technologie. Celery

Role. Exécution asynchrone des analyses et des reentrainements.

redis

Technologie. Redis

Role. Broker et backend de résultats Celery.

postgres_db

Technologie. PostgreSQL

Role. Stockage des entreprises, runs, avis, prédictions et corrections.

mlflow

Technologie. MLflow

Role. Suivi des expérimentations et registre de modèle.

worker

Technologie. Python

Role. Scripts historiques de scraping, ETL et entraînement.

dashboard

Technologie. Streamlit

Role. Ancienne interface d'exploration, conservée comme support historique.

Flux principal :

Utilisateur

-> React

-> FastAPI

-> PostgreSQL

- > Celery / Redis
- > Scraping Trustpilot ou import CSV
- > Prediction sentiment via modele MLflow
- > PostgreSQL
- > Rapport React, export CSV/PDF, corrections humaines

3. Sources de donnees

Deux modes d'acquisition sont disponibles.

3.1 Trustpilot

L'utilisateur saisit une entreprise ou une URL Trustpilot. L'API cree un run d'analyse et Celery execute le scraping avec Playwright. Le scraping peut cibler plusieurs pages par note, ce qui permet d'equilibrer les avis entre 1 et 5 etoiles.

Donnees collectees :

- auteur ;
- note ;
- date brute ;
- verbatim ;
- indicateur de reponse entreprise ;
- entreprise associee.

3.2 CSV utilisateur

L'utilisateur peut importer un fichier CSV d'avis. Ce mode permet d'analyser des exports clients ou des avis provenant d'autres plateformes sans dependre de Trustpilot.

Le controle avant import verifie :

- le nombre total d'avis ;
- le nombre de lignes exploitables ;
- le nombre de lignes ignorees ;
- les colonnes detectees ;
- un apercu des premiers verbatims.

Les colonnes minimales attendues sont :

- texte de l'avis ;
- note, si disponible.

Les colonnes optionnelles sont :

- auteur ;
- date ;
- reponse entreprise.

4. Modele de donnees

L'application produit utilise un schema PostgreSQL dedie.

companies

Role. Entreprises analysees et source associee.

analysis_runs

Role. Historique des analyses, statut, duree, source et fichiers produits.

analysis_run_events

Role. Journal d'execution lisible par l'utilisateur.

reviews

Role. Avis individuels rattaches a un run.

sentiment_predictions

Role. Prediction de sentiment et score du modele.

review_topics

Role. Irritants detectes sur chaque avis.

review_feedback

Role. Corrections humaines pour ameliorer le modele.

model_training_runs

Role. Historique des reentrainements lances depuis l'interface.

Ce modele separe les donnees brutes, les predictions, les corrections et les metadonnees d'execution. Il permet de conserver un historique auditable, de comparer plusieurs entreprises et de reutiliser les corrections pour le reentrainement.

Le schema produit est versionne avec Alembic. Une base existante non versionnee est validee puis rattachee a la baseline sans recreation des tables. Les tables historiques [dim_companies](#) et [fact_reviews](#) restent initialisees par [init_db.sql](#) pour les anciens scripts.

5. Pipeline d'analyse

- L'utilisateur cree une analyse depuis le frontend.
- FastAPI valide la demande et cree un [analysis_run](#).
- Si une analyse identique est deja active, l'API renvoie un conflit pour eviter les doublons.
- Le run est envoye dans Celery.
- Le worker collecte les avis ou relit le CSV importe.
- Les avis sont nettoyes et normalises.
- Le modele de sentiment charge depuis MLflow predit un label et un score.
- Des irritants metier sont detectes par analyse lexicale.
- Les resultats sont stockes dans PostgreSQL.
- Le frontend affiche le rapport, les avis, les exports et le journal d'execution.

Les statuts principaux d'un run sont :

- [pending](#) ;
- [running](#) ;
- [completed](#) ;
- [failed](#) ;
- [empty](#).

6. Modele de sentiment

Le modele est un classifieur supervise scikit-learn entraine sur des avis annotes.

Caracteristiques :

- vectorisation TF-IDF des verbatims ;
- prise en compte reduite de la note client ;
- classification en trois classes : [Negatif](#), [Neutre](#), [Positif](#) ;

- evaluation par accuracy, precision, recall et F1-score ;
- serialisation locale ;
- publication dans MLflow avec alias de production.

La note client est volontairement ponderee afin que le texte garde une influence centrale. Ce choix repond au besoin metier : detecter les cas ou la note et le verbatim racontent des choses differentes.

7. Boucle d'amelioration humaine

L'application permet de corriger manuellement le sentiment d'un avis. Les corrections sont stockees dans `review_feedback`, exportables en CSV et integrees au prochain reentrainement.

Le reentrainement :

- combine le corpus historique annote et les corrections humaines ;
- pondere davantage les corrections recentes ;
- cree un snapshot auditable du dataset d'entrainement ;
- evalue le modele sur un split stratifie ;
- peut enregistrer une nouvelle version candidate dans MLflow ;
- ne met a jour l'alias de production et ne resynchronise les predictions qu'apres promotion explicite.

Cette boucle transforme l'application en outil d'amelioration continue plutot qu'en simple dashboard statique.

8. API FastAPI

L'API expose les principaux endpoints suivants.

GET /health

Role. Verification de disponibilite.

POST /analysis-runs

Role. Creation d'une analyse Trustpilot.

POST /analysis-runs/preview-csv

Role. Controle avant import CSV.

POST /analysis-runs/import-csv

Role. Import et analyse d'un CSV.

GET /analysis-runs

Role. Historique des analyses.

GET /analysis-runs/{run_id}

Role. Detail d'un run.

POST /analysis-runs/{run_id}/execute

Role. Relance d'un run.

GET /analysis-runs/{run_id}/events

Role. Journal d'execution.

GET /analysis-runs/{run_id}/summary

Role. Rapport synthetique.

GET /analysis-runs/{run_id}/reviews

Role. Avis pages et filtrables.

POST /analysis-runs/{run_id}/reviews/{review_id}/feedback

Role. Correction humaine.

GET /analysis-runs/feedback/quality

Role. Synthèse qualité IA.

GET /analysis-runs/compare

Role. Benchmark multi-entreprises.

GET /model-training/overview

Role. Etat du modèle de production.

POST /model-training/runs

Role. Lancement d'un reentrainement.

Les endpoints métier sont protégés par des jetons JWT et des rôles, avec isolation par organisation. L'endpoint `/health` reste public afin de faciliter les sondes de disponibilité.

9. Interface React

Le frontend React/Vite/TypeScript est l'interface principale du produit. Il permet :

- de lancer une analyse Trustpilot ;
- d'importer un CSV avec contrôle avant import ;
- de suivre l'historique des runs ;
- de consulter le journal d'exécution ;
- de visualiser les KPIs d'un rapport entreprise ;
- de comparer plusieurs runs dans un benchmark ;
- de corriger les labels d'avis ;
- de piloter la qualité IA ;
- de déclencher un reentrainement ;
- d'exporter les avis, les corrections et les rapports.

L'interface vise un usage par des profils métier : responsable service client, responsable supply chain, analyste data ou direction.

10. Restitution métier

Un rapport entreprise contient :

- volume d'avis analyses ;
- note moyenne ;
- confiance IA moyenne ;
- répartition des sentiments ;
- répartition par note ;
- irritants principaux ;
- avis critiques ;
- incohérences note / texte ;
- score santé ;
- priorités recommandées ;
- points de vigilance.

Le benchmark multi-entreprises permet de comparer deux à quatre runs et de détecter :

- l'entreprise la plus à risque ;
- les irritants communs ;
- les irritants propres ;

- les differences de repartition negative ;
- les signaux metier prioritaires.

11. Dockerisation et reproductibilite

Le projet se lance avec Docker Compose. Les services applicatifs, la base de donnees, Redis, MLflow et le frontend sont definis dans [docker-compose.yml](#).

Commandes principales :

```
docker-compose up -d --build api celery_worker frontend
docker-compose up -d postgres_db mlflow redis
docker-compose logs -f api
docker-compose logs -f celery_worker
```

Cette organisation rend le projet reproductible sur une autre machine equipee de Docker.

12. CI/CD et qualite

La CI GitHub Actions execute les tests backend, les tests et le build frontend, puis construit l'image Docker. Un job [e2e-smoke](#) manuel valide un demarrage Compose isole et un import CSV de 15 avis. Le workflow de deploiement manuel vise une VM de test existante et verifie API et frontend. Les runs exacts et leurs limites sont dans les preuves DevOps.

Les validations disponibles incluent :

- build du frontend ;
- compilation Python de l'API ;
- verification [git diff --check](#) ;
- migrations Alembic sur bases PostgreSQL temporaires ;
- test de l'endpoint [/health](#) ;
- execution d'analyses Trustpilot ;
- import CSV ;
- correction humaine ;
- reentrainement ;
- export CSV et PDF.

Le script KPI extrait les temps de deploiement des runs GitHub Actions. La supervision de test archive un JSON et un Step Summary. Un heartbeat et un evenement Monitor [schedule](#) sont desormais prouves ; le job Monitor planifie a ete [skipped](#), donc la sonde VM planifiee active reste a demontrer.

13. Securite et cadre reglementaire

Mesures deja presentes :

- authentication JWT locale, roles et isolation par organisation ;
- endpoint de sante public limite ;
- variables d'environnement et secrets GitHub pour la configuration de test ;
- separation des services Docker ;
- export explicite des donnees.

Points a renforcer pour une production reelle :

- remplacer les secrets de developpement par un gestionnaire de secrets ;
- renforcer l'authentification et la gestion des secrets pour une production reelle ;
- documenter la duree de conservation des avis ;

- anonymiser les auteurs si le contexte d'usage l'exige ;
- formaliser les contraintes RGPD.

14. Limites connues

- Le scraping Trustpilot dépend de la structure HTML du site.
- La tentative réelle versionnée a reçu HTTP 403 et n'a extrait aucun avis ; la preuve d'une collecte directe de plus de 10 000 avis manque.
- Les avis ironiques ou très courts restent difficiles à classer.
- La classe **Neutre** est plus difficile à apprendre.
- Les corrections humaines doivent rester cohérentes pour ne pas dégrader le modèle.
- Le monitoring GitHub Actions concerne la VM de test, pas un SLA de production ; des événements **schedule** sont prouvés, mais pas encore une sonde planifiée active de la VM.
- Le déploiement automatisé cible une VM scolaire existante, sans provisioning cloud.
- Le rollback DB/migrations n'a pas été démontré.

15. Perspectives

Les évolutions les plus pertinentes sont :

- compléter les preuves de volumétrie et de représentativité de collecte ;
- enrichir le corpus avec plus d'entreprises et de secteurs ;
- ajouter une classification thématique plus robuste ;
- brancher d'autres sources d'avis via API ou CSV ;
- vérifier une sonde planifiée active avant de revenir au cron horaire ;
- préparer un éventuel provisioning cloud distinct de la VM scolaire ;
- formaliser RGPD, rétention et sécurité de production.

16. Conclusion technique

Le projet couvre les principales étapes attendues : collecte, organisation, traitement, machine learning, restitution, API, dockerisation et boucle d'amélioration. Il a évolué d'un pipeline de scraping vers une application produite exploitable par une entreprise pour analyser sa satisfaction client et prioriser ses actions.

Annexe — précisions de traçabilité pour la soutenance

- **MLflow** : le registre conservé contient le modèle v53 promu et son run documenté. Un cold start sur un registre vierge enregistre le pickle versionné comme première version de ce registre (v1), sans recréer l'historique v1–v53. Les captures finales du registre réel restent à exporter ; voir [docs/MLFLOW_V53_PREUVES.md](#).
- **Scheduler** : le heartbeat planifié 35765192366 a réussi. Le run Monitor planifié 35764642522 a été créé, mais son job a été **skipped** ; cela ne prouve pas une sonde VM planifiée active.
- **Rollback** : retour applicatif manuel testé entre deux SHA, dont le second était un commit vide. Aucun retour de fonctionnalités distinctes ni downgrade des migrations PostgreSQL n'a été démontré.
- **Collecte** : la tentative Showroomprivé s'est arrêtée sur HTTP 403 avec zéro avis extrait ; l'exigence d'une collecte exhaustive d'une entreprise de plus de 10 000 avis reste NON CONFORME.